Homework - 8

Repository: https://github.com/Ani2508/data260-3209
Domain: Clinical Trial Listing
Tagged commit (hw8): be247b56e8ecf3a12af8762ef748d5e1cde4dc08

Section 0 - Personal Configuration

Value	Result
SID4	3209
PORT_BASE	8509 (8000 + 3209 mod 900)
PREFIX	s3209
SEED	3209
VERIFY_SEED	263209
DOMAIN_ID	1 - Clinical trial listings

Environment

Item	Value
Hardware	Apple MacBook Pro, Apple M5 Pro chip
RAM	48 GB
OS	macOS
Kafka	apache/kafka 3.8.0 (KRaft, single node) in Docker
Python client	kafka-python
Local model	llama3.1:8b (Ollama), Part 2 agents

Part 1 - Trial Service over Kafka

Trial logic runs only in the worker. Client and FastAPI only send and read Kafka messages.

I. Operations

Operation	Input data	Success output
CREATE_TRIAL	trialTitle, sponsor, email, content, status, terms	success: true, trial_id (uuid4), message: "Trial created"
GET_TRIAL	trial_id	success: true, trial: {trial_id, ...fields}

- **Validation:** trialTitle 5-200 chars, valid email, content > 25 chars, terms = true.
- **Status:** status in Recruiting / Active - Not Recruiting / Completed / Terminated.
- **Storage:** plain dict in the worker, keyed by trial_id.

Code Snippet:

```python
def validate_trial(data) -> list[str]:
    """Return a list of field errors (empty list = valid)."""
    if not isinstance(data, dict):
        return ["data must be a JSON object"]
    errors = [f"unknown field '{k}'" for k in data if k not in FIELDS]
    for key in ("trialTitle", "sponsor", "email", "content", "status"):
        value = data.get(key)
        if not isinstance(value, str) or not value.strip():
            errors.append(f"'{key}' is required")
    if errors:
        return errors
    if not 5 <= len(data["trialTitle"].strip()) <= 200:
        errors.append("'trialTitle' must be 5-200 characters")
    if len(data["sponsor"].strip()) > 200:
        errors.append("'sponsor' must be at most 200 characters")
    if not EMAIL_RE.match(data["email"].strip()):
        errors.append("'email' must be a valid email address")
    if len(data["content"].strip()) <= 25:
        errors.append("'content' must be more than 25 characters")
    if data["status"] not in STATUSES:
        errors.append(f"'status' must be one of {sorted(STATUSES)}")
    if data.get("terms") is not True:
        errors.append("'terms' must be true")
    return errors
```

Validation

Code Snippet:

```python
def handle_request(store: dict, request) -> dict:
    """Process one request message and return the response (always echoes correlation_id)."""
    cid = request.get("correlation_id") if isinstance(request, dict) else None
    op = request.get("operation") if isinstance(request, dict) else None

    def fail(error: str) -> dict:
        return {"correlation_id": cid, "operation": op, "success": False, "error": error}

    if not isinstance(cid, str) or not cid:
        return fail("Missing correlation_id")
    data = request.get("data")

    if op == "CREATE_TRIAL":
        errors = validate_trial(data)
        if errors:
            return fail("Validation failed: " + "; ".join(errors))
        trial_id = str(uuid.uuid4())
        store[trial_id] = {"trial_id": trial_id, **{k: data[k] for k in FIELDS}}
        return {"correlation_id": cid, "operation": op, "success": True,
                "trial_id": trial_id, "message": "Trial created"}

    if op == "GET_TRIAL":
        trial_id = data.get("trial_id") if isinstance(data, dict) else None
        if not isinstance(trial_id, str) or not trial_id.strip():
            return fail("'trial_id' is required")
        if trial_id not in store:
            return fail("Trial not found")
        return {"correlation_id": cid, "operation": op, "success": True,
                "trial": store[trial_id]}

    return fail("Unknown operation")
```

Request handler

II. Kafka Topics and Message Schemas

Topic	Producer	Consumer
s3209_trial_requests	client / FastAPI	worker (group s3209_trial_worker)
s3209_trial_responses	worker (topic from reply_to)	client / FastAPI

Message	Fields
Request	correlation_id, reply_to = s3209_trial_responses, operation, data
Response (ok)	correlation_id, operation, success: true, trial_id or trial
Response (error)	correlation_id, operation, success: false, error

correlation_id flow

- Client creates one uuid4 per request, before sending.
- Sent as the message field and the Kafka key.
- Worker copies it into every response, errors included.
- Client matches the reply on it; FastAPI passes X-Correlation-ID through.

Code Snippet:

```
def process(msg, producer):
    request = parse(msg.value)
    cid = request.get("correlation_id") if isinstance(request, dict) else None
    op = request.get("operation") if isinstance(request, dict) else None
    log.info("RECEIVED  from %s [p%d@%d] cid=%s op=%s", msg.topic, msg.partition,
             msg.offset, cid, op)
    if request is None:
        response = {"correlation_id": None, "success": False, "error": "Invalid JSON"}
    else:
        response = handle_request(TRIALS, request)
    log.info("PROCESSED cid=%s success=%s %s", cid, response["success"],
             response.get("trial_id") or response.get("error") or "")

    reply_to = request.get("reply_to") if isinstance(request, dict) else None
    if not isinstance(reply_to, str) or not reply_to.startswith(f"{PREFIX}_"):
        reply_to = RESPONSE_TOPIC
    producer.send(reply_to, key=cid or "none", value=response).get(timeout=10)
    log.info("REPLIED   to %s cid=%s %s", reply_to, cid, json.dumps(response))


def main():
    # max_poll_records=1 + manual commit: the offset moves only after the reply is written
    consumer = KafkaConsumer(REQUEST_TOPIC, bootstrap_servers=BOOTSTRAP, group_id=WORKER_GROUP,
                             enable_auto_commit=False, auto_offset_reset="earliest",
                             max_poll_records=1)
    producer = KafkaProducer(bootstrap_servers=BOOTSTRAP, acks="all",
                             key_serializer=str.encode,
                             value_serializer=lambda v: json.dumps(v).encode())
    log.info("Run by: %s (%s)   bootstrap=%s", RUN_BY, PREFIX, BOOTSTRAP)
    ready = False
    while True:
        batch = consumer.poll(timeout_ms=1000)
        if not ready and consumer.assignment():
            log.info("Consumer is ready and listening... topic=%s group=%s",
                     REQUEST_TOPIC, WORKER_GROUP)
            ready = True
        for records in batch.values():
            for msg in records:
                process(msg, producer)
                consumer.commit()
```

Worker loop

Code Snippet:

```
def send(self, operation: str, data: dict, correlation_id: str | None = None) -> dict:
    """Publish one request message; returns the message that was sent."""
    cid = correlation_id or self.new_correlation_id()
    message = {"correlation_id": cid, "reply_to": RESPONSE_TOPIC,
               "operation": operation, "data": data}
    with self._lock:
        self._waiting[cid] = {"event": threading.Event(), "response": None}
    self.producer.send(REQUEST_TOPIC, key=cid, value=message).get(timeout=10)
    self.log.info("SENT      to %s cid=%s %s", REQUEST_TOPIC, cid, json.dumps(message))
    return message

def wait(self, message: dict, timeout: float = 15.0) -> dict:
    """Block until the response with this message's correlation_id arrives."""
    cid = message["correlation_id"]
    with self._lock:
        slot = self._waiting[cid]
    try:
        if not slot["event"].wait(timeout):
            return {"correlation_id": cid, "operation": message["operation"],
                    "success": False, "error": f"No response within {timeout:.0f}s"}
        response = slot["response"]
        self.log.info("RESPONSE  from %s cid=%s %s", RESPONSE_TOPIC, cid,
                      json.dumps(response))
        return response
    finally:
        with self._lock:
            self._waiting.pop(cid, None)
```

Kafka client

Code Snippet:

```python
@asynccontextmanager
async def lifespan(_app: FastAPI):
    global client
    client = TrialKafkaClient(role="api")
    log.info("Run by: %s (%s)", RUN_BY, PREFIX)
    log.info("Server started at http://127.0.0.1:%d", PORT_BASE)
    yield
    client.close()


app = FastAPI(title=f"{PREFIX} trial service (Kafka)", lifespan=lifespan)


def reply(response: dict) -> JSONResponse:
    if response["success"]:
        code = 200
    elif response.get("error") == "Trial not found":
        code = 404
    elif str(response.get("error", "")).startswith("No response"):
        code = 504
    else:
        code = 422
    return JSONResponse(response, status_code=code,
                        headers={"X-Correlation-ID": str(response.get("correlation_id"))})


@app.post("/trials")
def create_trial(data: dict, x_correlation_id: str | None = Header(default=None)):
    cid = x_correlation_id or str(uuid.uuid4())
    log.info("HTTP POST /trials cid=%s", cid)
    return reply(client.request("CREATE_TRIAL", data, correlation_id=cid))


@app.get("/trials/{trial_id}")
def get_trial(trial_id: str, x_correlation_id: str | None = Header(default=None)):
    cid = x_correlation_id or str(uuid.uuid4())
    log.info("HTTP GET /trials/%s cid=%s", trial_id, cid)
    return reply(client.request("GET_TRIAL", {"trial_id": trial_id}, correlation_id=cid))
```

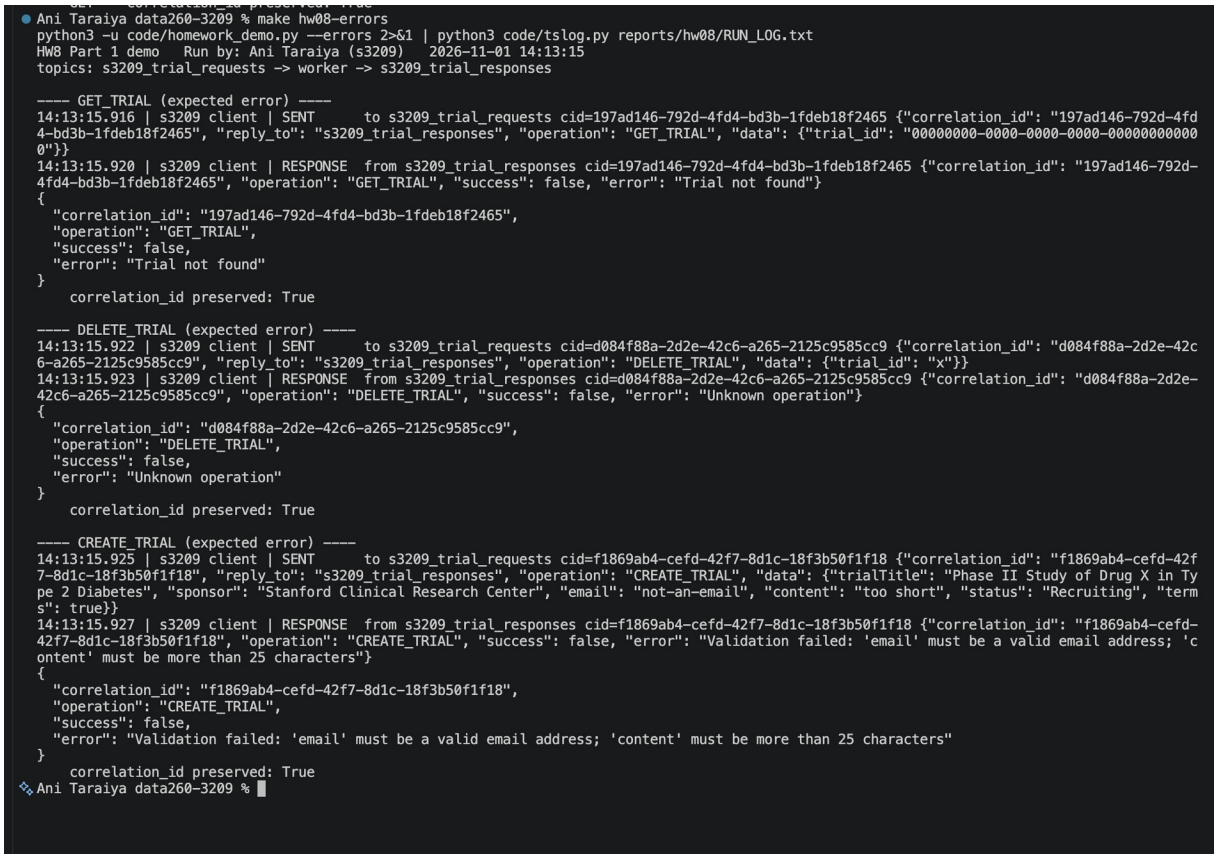
FastAPI wrapper

III. Error Handling

Case	Response
Missing / empty / invalid field	success: false, error: "Validation failed: ..."
Unknown operation	success: false, error: "Unknown operation"
Unknown trial_id	success: false, error: "Trial not found"

Every error keeps the original correlation_id.

Output Snippet:



Error responses

IV. Demonstration

Code Snippet:

```python
def main():
    print(f"HW8 Part 1 demo   Run by: {RUN_BY} ({PREFIX})   "
          f"{datetime.now():%Y-%m-%d %H:%M:%S}")
    print(f"topics: {REQUEST_TOPIC} -> worker -> {RESPONSE_TOPIC}")
    client = TrialKafkaClient(role="client")
    if "--errors" in sys.argv:
        error_cases(client)
        client.close()
        return

    print("\n================ CREATE_TRIAL ================")
    cid = client.new_correlation_id()
    step(1, f"correlation_id created: {cid}")
    step(2, f"CREATE_TRIAL request sent to {REQUEST_TOPIC}")
    message = client.send("CREATE_TRIAL", TRIAL, correlation_id=cid)
    step(3, "message on the request topic")
    print(json.dumps(message, indent=2))
    created = client.wait(message)
    lines = worker_lines(cid)
    step(4, "worker received and processed the request")
    show(lines, "RECEIVED")
    show(lines, "PROCESSED")
    step(5, f"worker sent the response to {RESPONSE_TOPIC}")
    show(lines, "REPLIED")
    step(6, "response received by the client")
    print(json.dumps(created, indent=2))
    if not created.get("success"):
        sys.exit("CREATE_TRIAL failed")
    trial_id = created["trial_id"]
    step(7, f"trial_id extracted: {trial_id}")

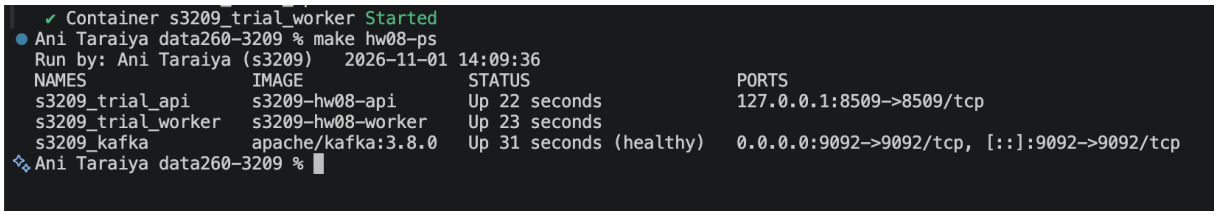
    print("\n================ GET_TRIAL ================")
    get_cid = client.new_correlation_id()
    step(8, f"GET_TRIAL request sent with trial_id {trial_id} (correlation_id {get_cid})")
    got = client.request("GET_TRIAL", {"trial_id": trial_id}, correlation_id=get_cid)
    step(9, "GET_TRIAL sequence: send -> receive -> process -> reply -> client response")
    print(f"    SEND      client -> {REQUEST_TOPIC} cid={get_cid}")
    show(worker_lines(get_cid), "")
    print(f"    RESPONSE  {RESPONSE_TOPIC} -> client cid={got.get('correlation_id')} "
          f"success={got.get('success')}")

    step(10, "retrieved trial")
    print(json.dumps(got.get("trial"), indent=2))
    same = bool(got.get("success")) and {k: got["trial"][k] for k in TRIAL} == TRIAL
    print(f"    trial data matches what was created: {same}")
    print(f"    CREATE correlation_id preserved: {created['correlation_id'] == cid}")
    print(f"    GET    correlation_id preserved: {got.get('correlation_id') == get_cid}")
    client.close()
```

Demo script

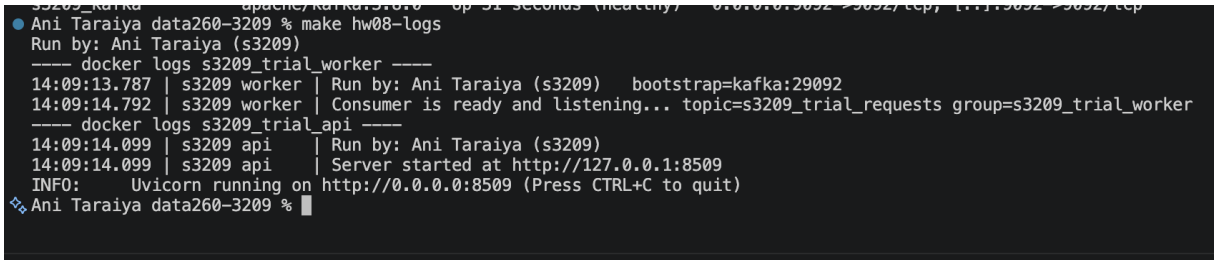
V. Evidence

Output Snippet:



docker ps

Output Snippet:



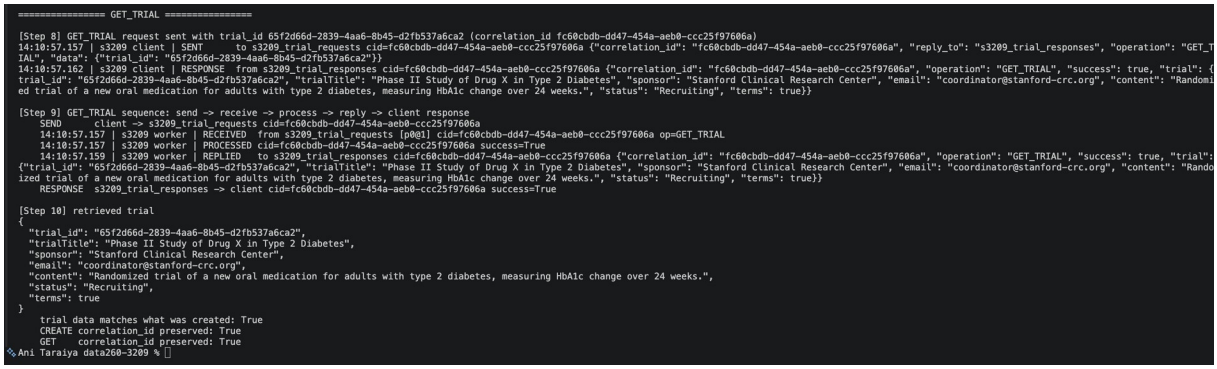
Startup logs

Output Snippet:



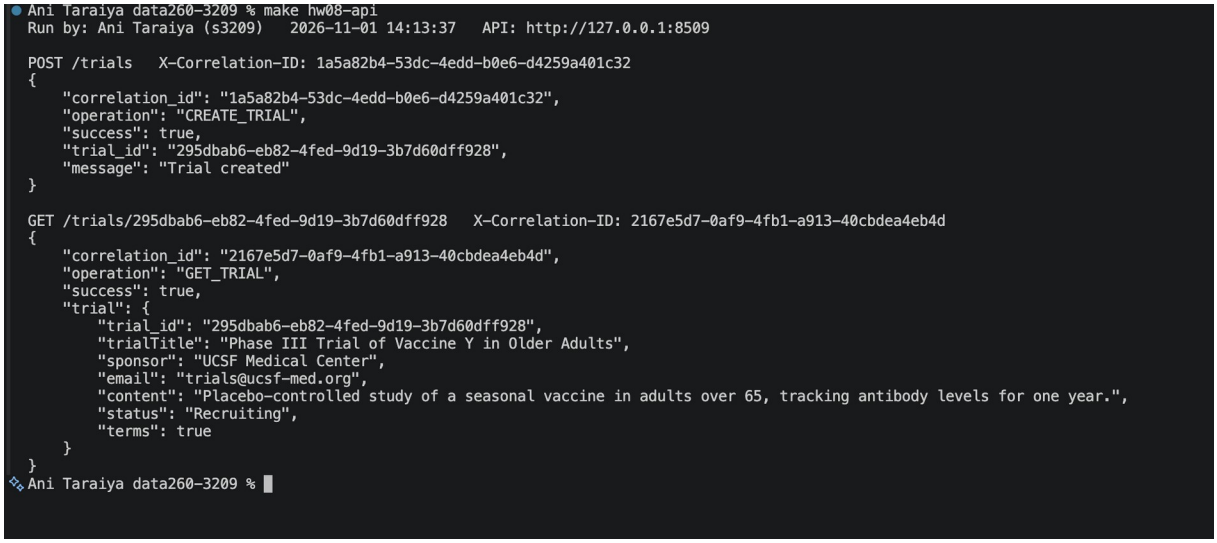
CREATE flow

Output Snippet:



GET flow

Output Snippet:



API flow

Part 2 - Three-Agent Kafka Pipeline

Planner, Writer and Reviewer run as separate processes. They never call each other; every hand-off is a Kafka message keyed by correlation_id.

I. Topics and Workflow

Sender	Reads	Sends to	Partitions
You (send_question.py)	-	s3209_inbox (question)	1
Planner	s3209_inbox	s3209_tasks (plan)	2
Writer	s3209_tasks	s3209_drafts (draft)	1
Reviewer	s3209_drafts	s3209_final (status + answer)	1

Output Snippet:

```
Ani Taraiya data260-3209 % make hw08-topics-delete
clear
make hw08-topics
$ kafka-topics.sh --create --if-not-exists --topic s3209_tasks --partitions 2 --replication-factor 1
Created topic s3209_tasks.
$ kafka-topics.sh --create --if-not-exists --topic s3209_drafts --partitions 1 --replication-factor 1
Created topic s3209_drafts.
$ kafka-topics.sh --create --if-not-exists --topic s3209_final --partitions 1 --replication-factor 1
Created topic s3209_final.

$ kafka-topics.sh --describe
Topic: s3209_inbox PartitionCount: 1 ReplicationFactor: 1
Topic: s3209_tasks PartitionCount: 2 ReplicationFactor: 1
Topic: s3209_drafts PartitionCount: 1 ReplicationFactor: 1
Topic: s3209_final PartitionCount: 1 ReplicationFactor: 1
Ani Taraiya data260-3209 %
```

Topic creation

II. Agents

- **Groups:** one consumer group per agent (s3209_planner, s3209_writer, s3209_reviewer).
- **Consume:** max_poll_records = 1, auto-commit off.
- **Commit:** offset committed only after the next message is acked.
- **Model:** llama3.1:8b through the HW1 model adapter, temperature 0, seed 3209.

Code Snippet:

```python

def run_agent(name: str, in_topic: str, out_topic: str, handle):
    """Consume in_topic one message at a time, send handle(msg) to out_topic, then commit."""
    consumer = KafkaConsumer(in_topic, bootstrap_servers=BOOTSTRAP,
                             group_id=f"{PREFIX}_{name.lower()}", enable_auto_commit=False,
                             auto_offset_reset="earliest", max_poll_records=1)
    out = producer()
    log(name, f"Run by: {RUN_BY} ({PREFIX})   model={MODEL}   {in_topic} -> {out_topic}")
    ready = False
    while True:
        batch = consumer.poll(timeout_ms=1000)
        if not ready and consumer.assignment():
            log(name, f"Consumer is ready and listening... topic={in_topic}")
            ready = True
        for records in batch.values():
            for rec in records:
                msg = json.loads(rec.value)
                cid = msg.get("correlation_id")
                log(name, f"RECEIVED from {in_topic} [p{rec.partition}@{rec.offset}] cid={cid}")
                result = handle(msg)
                out.send(out_topic, key=cid, value=result).get(timeout=10)
                log(name, f"SENT     to {out_topic} cid={cid} {json.dumps(result)[:160]}")
                consumer.commit()
                log(name, f"COMMIT   {in_topic} p{rec.partition} offset -> {rec.offset + 1}")

```

Agent loop

Code Snippet:

```
code > kafka_agents > planner.py > ...
"""HW8 Part 2 Planner: s3209_inbox (question) -> s3209_tasks (plan)."""
from common import INBOX, TASKS, ask_json, log, model, now, run_agent

SYSTEM = ("You are the Planner for a clinical trial listings help desk. Break the user's "
          "question into exactly 3 short steps a writer should follow to answer it. "
          'Reply only with JSON: {"plan": ["step 1", "step 2", "step 3"]}')
llm = model()

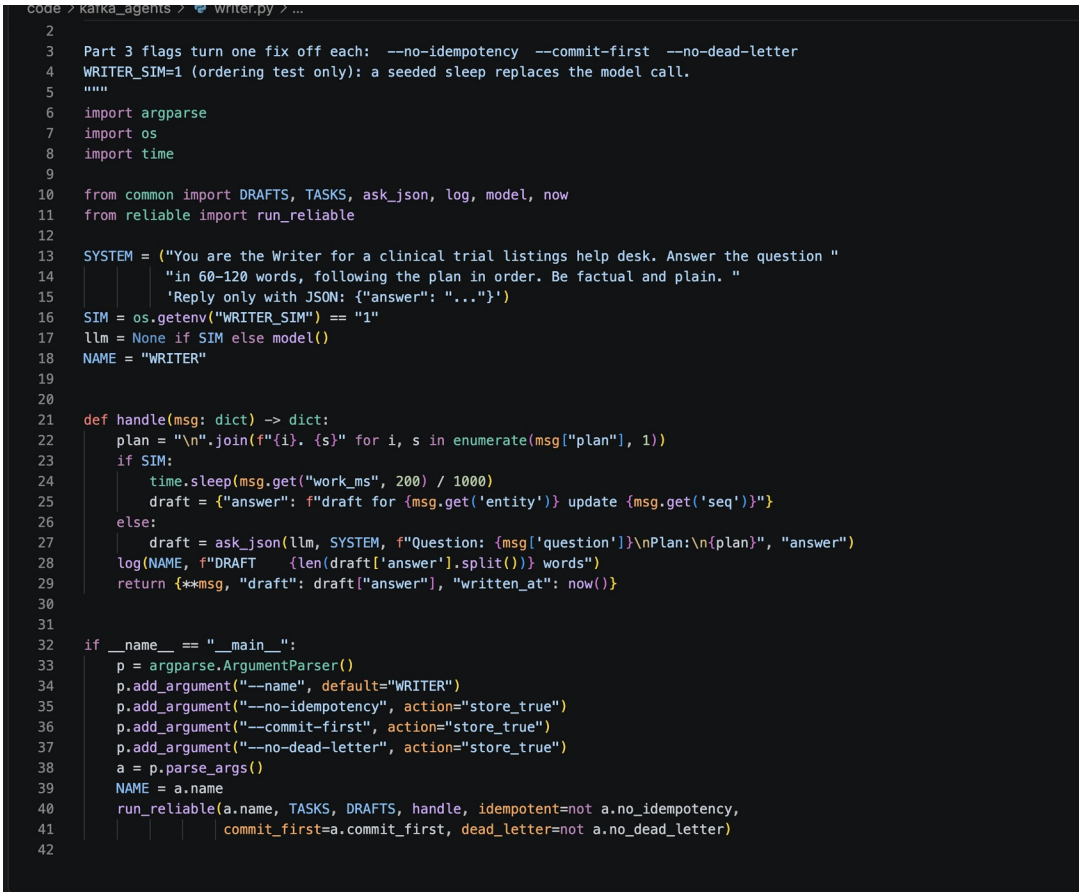

def handle(msg: dict) -> dict:
    plan = ask_json(llm, SYSTEM, msg["question"], "plan")["plan"][:3]
    log("PLANNER", f"PLAN     {plan}")
    return {"correlation_id": msg["correlation_id"], "question": msg["question"],
            "plan": plan, "planned_at": now()}


if __name__ == "__main__":
    run_agent("PLANNER", INBOX, TASKS, handle)

```

Planner

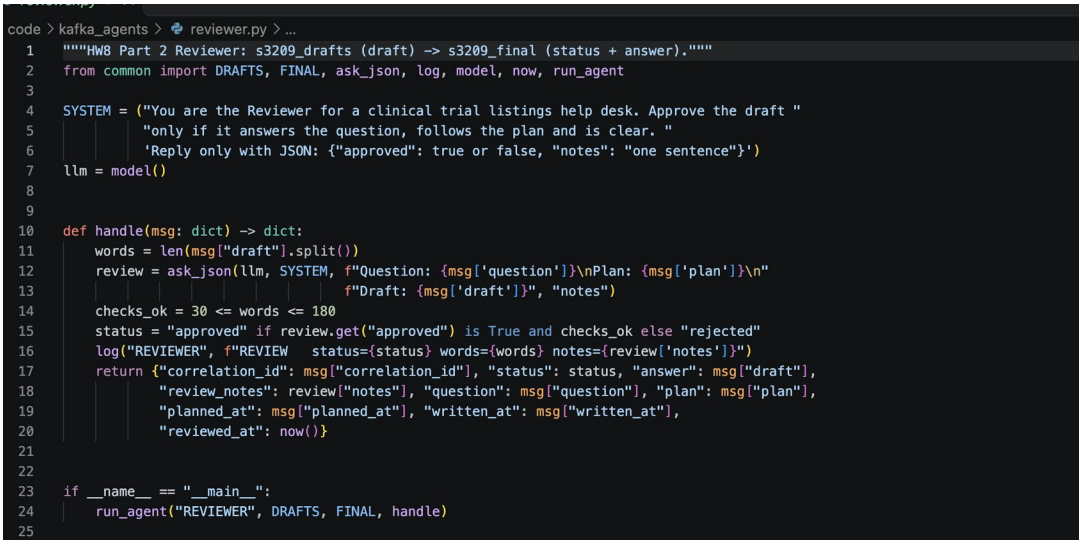
Code Snippet:



```
code > kafka_agents > writer.py > ...
  2
  3   Part 3 flags turn one fix off each:  --no-idempotency  --commit-first  --no-dead-letter
  4   WRITER_SIM=1 (ordering test only): a seeded sleep replaces the model call.
  5   """
  6   import argparse
  7   import os
  8   import time
  9
 10   from common import DRAFTS, TASKS, ask_json, log, model, now
 11   from reliable import run_reliable
 12
 13   SYSTEM = ("You are the Writer for a clinical trial listings help desk. Answer the question "
 14             "in 60-120 words, following the plan in order. Be factual and plain. "
 15             'Reply only with JSON: {"answer": "..."}')
 16   SIM = os.getenv("WRITER_SIM") == "1"
 17   llm = None if SIM else model()
 18   NAME = "WRITER"
 19
 20
 21   def handle(msg: dict) -> dict:
 22       plan = "\n".join(f"{i}. {s}" for i, s in enumerate(msg["plan"], 1))
 23       if SIM:
 24           time.sleep(msg.get("work_ms", 200) / 1000)
 25           draft = {"answer": f"draft for {msg.get('entity')} update {msg.get('seq')}"}
 26       else:
 27           draft = ask_json(llm, SYSTEM, f"Question: {msg['question']}\nPlan:\n{plan}", "answer")
 28       log(NAME, f"DRAFT    {len(draft['answer'].split())} words")
 29       return {**msg, "draft": draft["answer"], "written_at": now()}
 30
 31
 32   if __name__ == "__main__":
 33       p = argparse.ArgumentParser()
 34       p.add_argument("--name", default="WRITER")
 35       p.add_argument("--no-idempotency", action="store_true")
 36       p.add_argument("--commit-first", action="store_true")
 37       p.add_argument("--no-dead-letter", action="store_true")
 38       a = p.parse_args()
 39       NAME = a.name
 40       run_reliable(a.name, TASKS, DRAFTS, handle, idempotent=not a.no_idempotency,
 41                    commit_first=a.commit_first, dead_letter=not a.no_dead_letter)
 42
```

Writer

Code Snippet:



```
code > kafka_agents > reviewer.py > ...
  1   """HW8 Part 2 Reviewer: s3209_drafts (draft) -> s3209_final (status + answer)."""
  2   from common import DRAFTS, FINAL, ask_json, log, model, now, run_agent
  3
  4   SYSTEM = ("You are the Reviewer for a clinical trial listings help desk. Approve the draft "
  5             "only if it answers the question, follows the plan and is clear. "
  6             'Reply only with JSON: {"approved": true or false, "notes": "one sentence"}')
  7   llm = model()
  8
  9
 10   def handle(msg: dict) -> dict:
 11       words = len(msg["draft"].split())
 12       review = ask_json(llm, SYSTEM, f"Question: {msg['question']}\nPlan: {msg['plan']}\n"
 13                                      f"Draft: {msg['draft']}", "notes")
 14       checks_ok = 30 <= words <= 180
 15       status = "approved" if review.get("approved") is True and checks_ok else "rejected"
 16       log("REVIEWER", f"REVIEW   status={status} words={words} notes={review['notes']}")
 17       return {"correlation_id": msg["correlation_id"], "status": status, "answer": msg["draft"],
 18               "review_notes": review["notes"], "question": msg["question"], "plan": msg["plan"],
 19               "planned_at": msg["planned_at"], "written_at": msg["written_at"],
 20               "reviewed_at": now()}
 21
 22
 23   if __name__ == "__main__":
 24       run_agent("REVIEWER", DRAFTS, FINAL, handle)
 25
```

Reviewer

Code Snippet:

```python
    log("YOU", f"Run by: {RUN_BY} ({PREFIX})")
    final = KafkaConsumer(bootstrap_servers=BOOTSTRAP, group_id=None, enable_auto_commit=False)
    tp = TopicPartition(FINAL, 0)
    final.assign([tp])
    final.seek_to_end(tp)
    final.position(tp)

    cid = str(uuid.uuid4())
    msg = {"correlation_id": cid, "question": question, "sent_at": now()}
    producer().send(INBOX, key=cid, value=msg).get(timeout=10)
    log("YOU", f"SENT     to {INBOX} cid={cid} {json.dumps(msg)}")

    deadline = time.time() + 300
    while time.time() < deadline:
        for records in final.poll(timeout_ms=1000).values():
            for rec in records:
                result = json.loads(rec.value)
                if result.get("correlation_id") != cid:
                    continue
                log("YOU", f"FINAL    from {FINAL} cid={cid} status={result['status']}")
                print(json.dumps(result, indent=2))
                print(f"correlation_id preserved: {result['correlation_id'] == cid}")
                RAW.mkdir(parents=True, exist_ok=True)
                (RAW / "part2_final.json").write_text(json.dumps(result, indent=2) + "\n")
                show_journey(cid)
                return
    sys.exit(f"no final message for {cid} within 300s")

```

Send question

III. Run

Terminal	Command
1	make hw08-planner (python planner.py)
2	make hw08-writer (python writer.py)
3	make hw08-reviewer (python reviewer.py)
4	make hw08-ask (python send_question.py)

Output Snippet:

```
Ani Taraiya data260-3209 % make hw08-planner
cd code/kafka_agents && python3 -u planner.py 2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
14:29:33.210 | s3209 PLANNER  | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_inbox -> s3209_tasks
14:29:34.212 | s3209 PLANNER  | Consumer is ready and listening... topic=s3209_inbox
14:42:46.359 | s3209 PLANNER  | RECEIVED from s3209_inbox [p0@0] cid=369a3d5c-273f-45b8-9288-1d197fa769f5
14:42:50.408 | s3209 PLANNER  | PLAN     ["Step 1: Review the trial's inclusion and exclusion criteria to ensure the
patient meets the necessary requirements.", "Step 2: Understand the trial's treatment and intervention, including any
 potential risks or side effects, to make an informed decision.", "Step 3: Discuss the trial with a healthcare provid
er to determine if it's a good fit for their specific needs and medical history."]
14:42:50.419 | s3209 PLANNER  | SENT     to s3209_tasks cid=369a3d5c-273f-45b8-9288-1d197fa769f5 {"correlation_id": "
369a3d5c-273f-45b8-9288-1d197fa769f5", "question": "What should a patient check before joining a recruiting Phase II
clinical trial for type
14:42:50.426 | s3209 PLANNER  | COMMIT   s3209_inbox p0 offset -> 1
14:45:01.993 | s3209 PLANNER  | RECEIVED from s3209_inbox [p0@1] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
14:45:03.717 | s3209 PLANNER  | PLAN     ["Step 1: Review the trial's inclusion and exclusion criteria to ensure the
patient meets the necessary requirements.", "Step 2: Understand the trial's treatment and intervention, including any
 potential risks or side effects, to make an informed decision.", "Step 3: Discuss the trial with a healthcare provid
er to determine if it's a good fit for their specific needs and medical history."]
14:45:03.721 | s3209 PLANNER  | SENT     to s3209_tasks cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa {"correlation_id": "
c5edba34-c6a6-4e2c-9a65-68f30373bdaa", "question": "What should a patient check before joining a recruiting Phase II
clinical trial for type
14:45:03.725 | s3209 PLANNER  | COMMIT   s3209_inbox p0 offset -> 2
```

Planner log

Output Snippet:

```
(.venv) anitaraiya@mac data260-3209 % make hw08-writer
cd code/kafka_agents && python3 -u writer.py 2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
14:39:47.376 | s3209 WRITER   | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
14:39:48.377 | s3209 WRITER   | Consumer is ready and listening... topic=s3209_tasks
14:42:50.419 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@0] cid=369a3d5c-273f-45b8-9288-1d197fa769f5
14:42:52.384 | s3209 WRITER   | DRAFT    74 words
14:42:52.390 | s3209 WRITER   | SENT     to s3209_drafts cid=369a3d5c-273f-45b8-9288-1d197fa769f5 {"correlation_id":
"369a3d5c-273f-45b8-9288-1d197fa769f5", "question": "What should a patient check before joining a recruiting Phase II
 clinical trial for type
14:42:52.393 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 1
14:45:03.722 | s3209 WRITER   | RECEIVED from s3209_tasks [p1@0] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
14:45:05.447 | s3209 WRITER   | DRAFT    74 words
14:45:05.450 | s3209 WRITER   | SENT     to s3209_drafts cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa {"correlation_id":
"c5edba34-c6a6-4e2c-9a65-68f30373bdaa", "question": "What should a patient check before joining a recruiting Phase II
 clinical trial for type
14:45:05.452 | s3209 WRITER   | COMMIT   s3209_tasks p1 offset -> 1
```

Writer log

Output Snippet:

```
(.venv) anitaraiya@mac data260-3209 % make hw08-reviewer
cd code/kafka_agents && python3 -u reviewer.py 2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
14:41:27.224 | s3209 REVIEWER | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_drafts -> s3209_final
14:41:28.225 | s3209 REVIEWER | Consumer is ready and listening... topic=s3209_drafts
14:42:52.389 | s3209 REVIEWER | RECEIVED from s3209_drafts [p0@0] cid=369a3d5c-273f-45b8-9288-1d197fa769f5
14:42:53.408 | s3209 REVIEWER | REVIEW   status=approved words=74 notes=The draft clearly answers the question and fo
llows the plan, providing a concise and informative response for patients considering a Phase II clinical trial for t
ype 2 diabetes.
14:42:53.416 | s3209 REVIEWER | SENT     to s3209_final cid=369a3d5c-273f-45b8-9288-1d197fa769f5 {"correlation_id": "
369a3d5c-273f-45b8-9288-1d197fa769f5", "status": "approved", "answer": "Before joining a Phase II clinical trial for
type 2 diabetes, a pati
14:42:53.420 | s3209 REVIEWER | COMMIT   s3209_drafts p0 offset -> 1
14:45:05.450 | s3209 REVIEWER | RECEIVED from s3209_drafts [p0@1] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
14:45:06.243 | s3209 REVIEWER | REVIEW   status=approved words=74 notes=The draft clearly answers the question and fo
llows the plan, providing a concise and informative response for patients considering a Phase II clinical trial for t
ype 2 diabetes.
14:45:06.247 | s3209 REVIEWER | SENT     to s3209_final cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa {"correlation_id": "
c5edba34-c6a6-4e2c-9a65-68f30373bdaa", "status": "approved", "answer": "Before joining a Phase II clinical trial for
type 2 diabetes, a pati
14:45:06.251 | s3209 REVIEWER | COMMIT   s3209_drafts p0 offset -> 2
```

Reviewer log

IV. Result

Output Snippet:

```
14:45:06.248 | s3209 YOU      | FINAL    from s3209_final cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa status=approved
{
  "correlation_id": "c5edba34-c6a6-4e2c-9a65-68f30373bdaa",
  "status": "approved",
  "answer": "Before joining a Phase II clinical trial for type 2 diabetes, a patient should review the trial's inclus
ion and exclusion criteria to ensure they meet the necessary requirements. They should also understand the trial's tr
eatment and intervention, including any potential risks or side effects, to make an informed decision. It's also reco
mmended to discuss the trial with a healthcare provider to determine if it's a good fit for their specific needs and
medical history.",
  "review_notes": "The draft clearly answers the question and follows the plan, providing a concise and informative r
esponse for patients considering a Phase II clinical trial for type 2 diabetes.",
  "question": "What should a patient check before joining a recruiting Phase II clinical trial for type 2 diabetes?",
  "plan": [
    "Step 1: Review the trial's inclusion and exclusion criteria to ensure the patient meets the necessary requiremen
ts.",
    "Step 2: Understand the trial's treatment and intervention, including any potential risks or side effects, to mak
e an informed decision.",
    "Step 3: Discuss the trial with a healthcare provider to determine if it's a good fit for their specific needs an
d medical history."
  ],
  "planned_at": "14:45:03.717",
  "written_at": "14:45:05.447",
  "reviewed_at": "14:45:06.244"
}
correlation_id preserved: True
```

Final message

Output Snippet:

```
  message journey for cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:01.994 | s3209 YOU      | SENT     to s3209_inbox cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:01.993 | s3209 PLANNER  | RECEIVED from s3209_inbox [p0@1] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:03.721 | s3209 PLANNER  | SENT     to s3209_tasks cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:03.722 | s3209 WRITER   | RECEIVED from s3209_tasks [p1@0] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:05.450 | s3209 WRITER   | SENT     to s3209_drafts cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:05.450 | s3209 REVIEWER | RECEIVED from s3209_drafts [p0@1] cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:06.247 | s3209 REVIEWER | SENT     to s3209_final cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa
    14:45:06.248 | s3209 YOU      | FINAL    from s3209_final cid=c5edba34-c6a6-4e2c-9a65-68f30373bdaa status=approved
Ani Taraiya data260-3209 %
```

Message journey

Stage	**Time**
Planner	1.73 s
Writer	1.73 s
Reviewer	0.80 s
Inbox to final	4.25 s

V. How Kafka Helped

- **Decoupling:** agents only know topic names, not each other.
- **Buffering:** a question waits in s3209_inbox if the Planner is down.
- **Tracing:** the same correlation_id keys every stage, so one grep shows the whole journey.
- **Scaling:** s3209_tasks has 2 partitions, so a second Writer can join the same group.

Part 3 - Failure Behavior

Each test starts its own Writer and runs twice: fix off (before) and fix on (after). Command: make hw08-<test>-before / -after. Raw results: reports/hw08/raw/part3_*.

Output Snippet:

```
Ani Taraiya data260-3209 % make hw08-topic-dlq
Run by: Ani Taraiya (s3209)
$ kafka-topics.sh --create --if-not-exists --topic s3209_dead_letter --partitions 1 --replication-factor 1
Topic: s3209_dead_letter PartitionCount: 1 ReplicationFactor: 1
Ani Taraiya data260-3209 %
```

Dead-letter topic

I. Duplicate Delivery

- **Test:** same message (same correlation_id) sent twice to s3209_tasks.
- **Fix:** a done-marker per correlation_id; a repeat is logged DUPLICATE and skipped.

Code Snippet:

```python
def process(name, rec, consumer, out, handle, done_dir, out_topic,
            idempotent, commit_first, max_retries, dead_letter):
    if commit_first:  # before fix: offset moves before the work is done
        commit(name, consumer, rec, " (before work)")
    try:
        msg = json.loads(rec.value)
    except ValueError:
        msg = None
    cid = msg.get("correlation_id") if isinstance(msg, dict) else None
    log(name, f"RECEIVED from {rec.topic} [p{rec.partition}@{rec.offset}] cid={cid}")

    marker = done_dir / f"{cid}.done"
    if idempotent and cid and marker.exists():  # idempotency: already handled
        log(name, f"DUPLICATE cid={cid} already processed -> skipped, nothing sent")
        commit(name, consumer, rec)
        return
```

Idempotency

Output Snippet:

```
Ani Taraiya data260-3209 %      make hw08-dup-before
cd code/kafka_agents && python3 -u failure_tests.py dup --before 2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LO
G.txt
15:19:58.688 | s3209 TEST     | Run by: Ani Taraiya (s3209)   test=dup   BEFORE fix
    15:19:58.981 | s3209 WRITER   | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
    15:19:58.981 | s3209 WRITER   | MODE     idempotency=OFF   commit=after success   dead_letter=on (3 tries)
    15:19:58.992 | s3209 WRITER   | ASSIGNED partitions=[]
    15:19:59.101 | s3209 WRITER   | ASSIGNED partitions=[0, 1]
15:19:59.645 | s3209 TEST     | SENT     to s3209_tasks [p0@40] cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65    15:19:59.
644 | s3209 WRITER   | Consumer is ready and listening... topic=s3209_tasks group=s3209_writer
    15:19:59.644 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@40] cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65

    15:20:02.528 | s3209 WRITER   | DRAFT    86 words
    15:20:02.539 | s3209 WRITER   | SENT     to s3209_drafts cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65
    15:20:02.542 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 41
15:20:02.679 | s3209 TEST     | REPLAY   same message again, same cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65
15:20:02.683 | s3209 TEST     | SENT     to s3209_tasks [p0@41] cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65
    15:20:02.683 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@41] cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65
    15:20:05.096 | s3209 WRITER   | DRAFT    86 words
    15:20:05.100 | s3209 WRITER   | SENT     to s3209_drafts cid=6c690851-d5bd-4f8d-8f7e-f04df9d9fb65
    15:20:05.104 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 42
15:20:19.462 | s3209 TEST     | RESULT   2 deliveries -> drafts written=2, final messages=2 (expected 1 each)
15:20:19.472 | s3209 TEST     | saved reports/hw08/raw/part3_dup_before.json
Ani Taraiya data260-3209 %
```

Duplicate before

Output Snippet:

```
Ani Taraiya data260-3209 % make hw08-dup-after
cd code/kafka_agents && python3 -u failure_tests.py dup  2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
15:20:39.352 | s3209 TEST     | Run by: Ani Taraiya (s3209)   test=dup   AFTER fix
    15:20:39.642 | s3209 WRITER   | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
    15:20:39.642 | s3209 WRITER   | MODE     idempotency=on   commit=after success   dead_letter=on (3 tries)
    15:21:02.119 | s3209 WRITER   | ASSIGNED partitions=[0, 1]
15:21:02.497 | s3209 TEST     | SENT     to s3209_tasks [p0@42] cid=91e48db3-3da8-4394-86ca-34e01c31dca2
    15:21:02.498 | s3209 WRITER   | Consumer is ready and listening... topic=s3209_tasks group=s3209_writer
    15:21:02.498 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@42] cid=91e48db3-3da8-4394-86ca-34e01c31dca2
    15:21:04.801 | s3209 WRITER   | DRAFT    86 words
    15:21:04.812 | s3209 WRITER   | SENT     to s3209_drafts cid=91e48db3-3da8-4394-86ca-34e01c31dca2
    15:21:04.818 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 43
15:21:04.933 | s3209 TEST     | REPLAY   same message again, same cid=91e48db3-3da8-4394-86ca-34e01c31dca2
15:21:04.935 | s3209 TEST     | SENT     to s3209_tasks [p0@43] cid=91e48db3-3da8-4394-86ca-34e01c31dca2
    15:21:04.935 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@43] cid=91e48db3-3da8-4394-86ca-34e01c31dca2
    15:21:04.936 | s3209 WRITER   | DUPLICATE cid=91e48db3-3da8-4394-86ca-34e01c31dca2 already processed -> skipped,
nothing sent
    15:21:04.937 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 44
15:21:19.579 | s3209 TEST     | RESULT   2 deliveries -> drafts written=1, final messages=1 (expected 1 each)
15:21:19.589 | s3209 TEST     | saved reports/hw08/raw/part3_dup_after.json
Ani Taraiya data260-3209 %
```

Duplicate after

II. Consumer Death

- **Test:** Writer killed with kill -9 while drafting, then restarted.
- **Fix:** commit the offset only after the draft is acked (not before work).

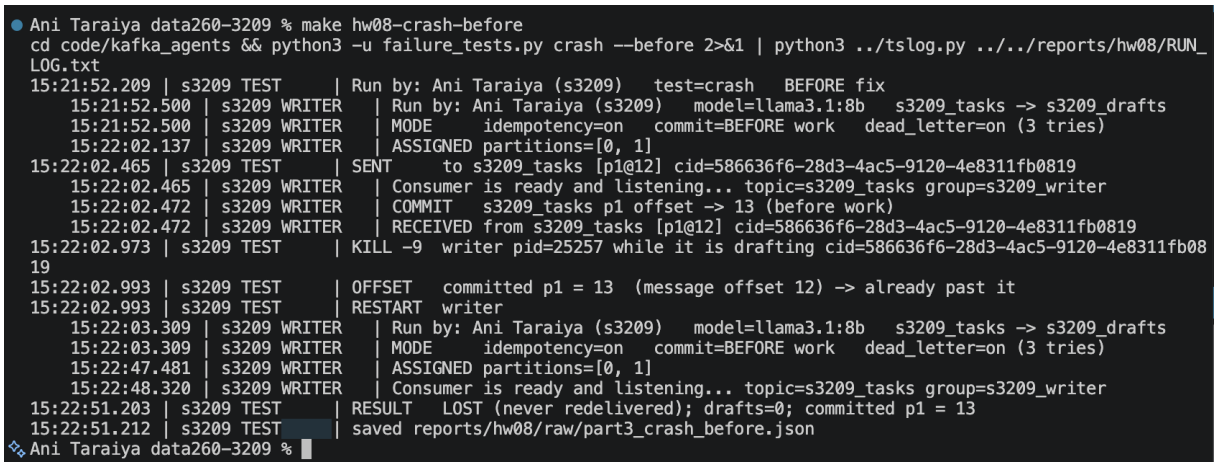
Code Snippet:

```python
for attempt in range(1, max_retries + 1):
    try:
        if not isinstance(msg, dict):
            raise ValueError("message is not a JSON object")
        result = handle(msg)
        break
    except Exception as exc:
        error = f"{type(exc).__name__}: {exc}"
        if not dead_letter:  # before fix: the error kills the consumer
            log(name, f"CRASH    cid={cid} {error} (offset not committed)")
            raise
        log(name, f"RETRY    {attempt}/{max_retries} cid={cid} {error}")
        time.sleep(0.5)
else:
    bad = {"correlation_id": cid, "error": error, "attempts": max_retries,
           "source": f"{rec.topic} p{rec.partition}@{rec.offset}",
           "raw": rec.value.decode(errors="replace"), "failed_at": now()}
    out.send(DEAD_LETTER, key=cid or "unknown", value=bad).get(timeout=10)
    log(name, f"DEAD_LETTER sent to {DEAD_LETTER} cid={cid} after {max_retries} tries")
    if not commit_first:
        commit(name, consumer, rec)
    return

result.update(processed_by=name, source_partition=rec.partition)
out.send(out_topic, key=cid, value=result).get(timeout=10)
log(name, f"SENT     to {out_topic} cid={cid}")
if idempotent and cid:
    marker.touch()
if not commit_first:
    commit(name, consumer, rec)
```

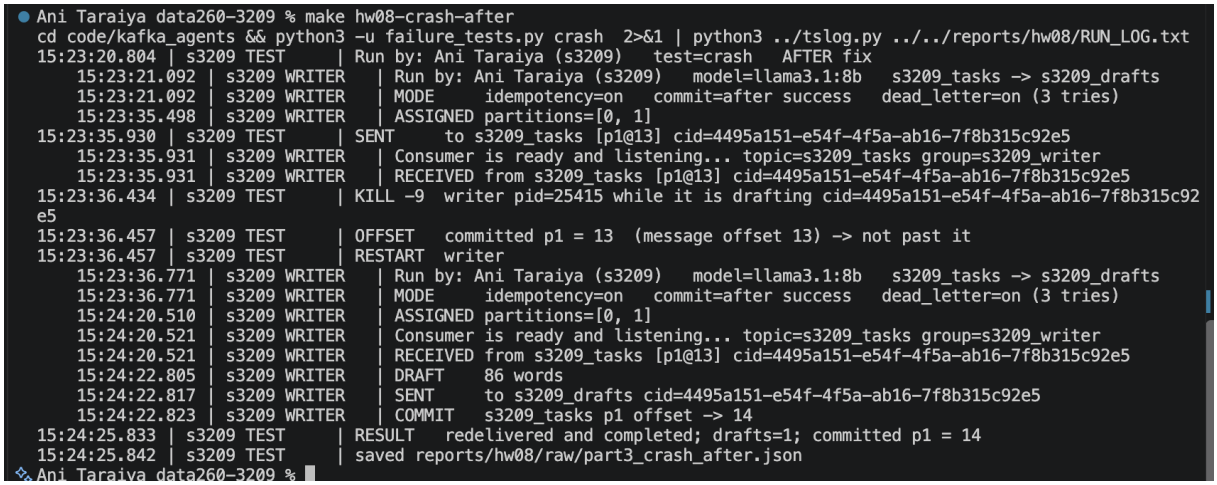
Retry + commit

Output Snippet:



Crash before

Output Snippet:

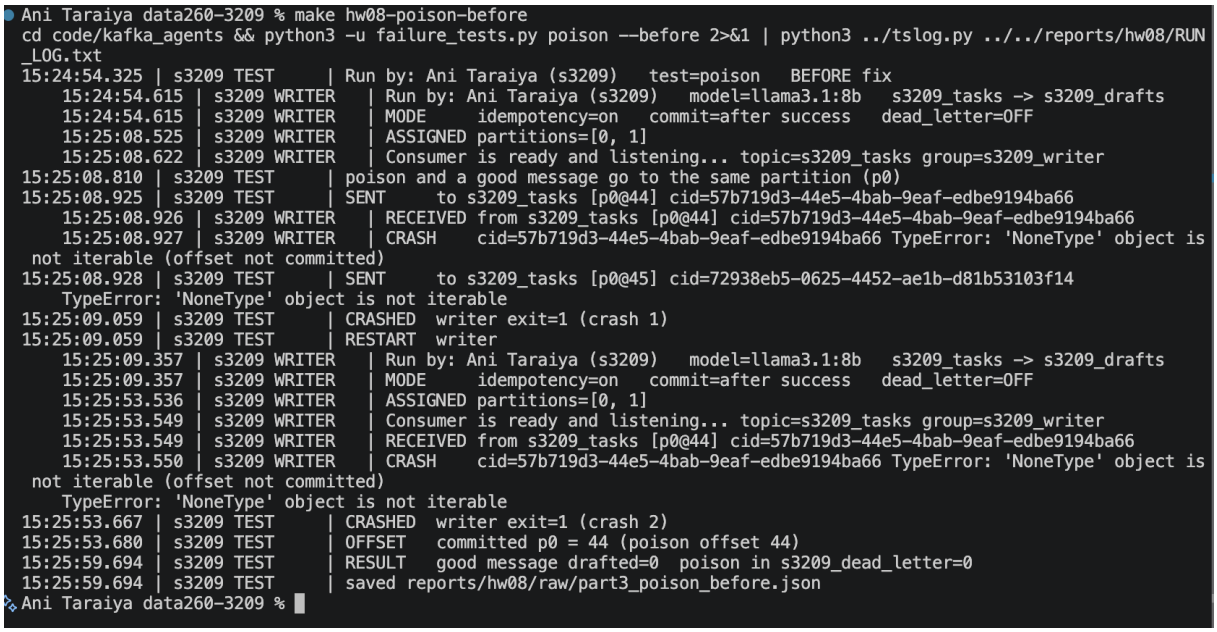


Crash after

III. Poison Message

- **Test:** a task with plan = null, then a good task, both on p0.
- **Fix:** 3 tries, then s3209_dead_letter, commit, move on.

Output Snippet:



Poison before

Output Snippet:

```
Ani Taraiya data260-3209 % make hw08-poison-after
cd code/kafka_agents && python3 -u failure_tests.py poison  2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
15:27:00.167 | s3209 TEST     | Run by: Ani Taraiya (s3209)   test=poison   AFTER fix
    15:27:00.456 | s3209 WRITER   | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
    15:27:00.456 | s3209 WRITER   | MODE     idempotency=on   commit=after success   dead_letter=on (3 tries)
    15:27:00.466 | s3209 WRITER   | ASSIGNED partitions=[]
    15:27:00.576 | s3209 WRITER   | ASSIGNED partitions=[0, 1]
    15:27:00.585 | s3209 WRITER   | Consumer is ready and listening... topic=s3209_tasks group=s3209_writer
    15:27:00.585 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@44] cid=57b719d3-44e5-4bab-9eaf-edbe9194ba66
    15:27:00.585 | s3209 WRITER   | RETRY    1/3 cid=57b719d3-44e5-4bab-9eaf-edbe9194ba66 TypeError: 'NoneType' objec
t is not iterable
15:27:01.020 | s3209 TEST     | poison and a good message go to the same partition (p0)
    15:27:01.088 | s3209 WRITER   | RETRY    2/3 cid=57b719d3-44e5-4bab-9eaf-edbe9194ba66 TypeError: 'NoneType' objec
t is not iterable
15:27:01.136 | s3209 TEST     | SENT     to s3209_tasks [p0@46] cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa
15:27:01.139 | s3209 TEST     | SENT     to s3209_tasks [p0@47] cid=ae7bff49-09c2-4920-92c1-a8b6320e0961
    15:27:01.589 | s3209 WRITER   | RETRY    3/3 cid=57b719d3-44e5-4bab-9eaf-edbe9194ba66 TypeError: 'NoneType' objec
t is not iterable
    15:27:02.096 | s3209 WRITER   | DEAD_LETTER sent to s3209_dead_letter cid=57b719d3-44e5-4bab-9eaf-edbe9194ba66 af
ter 3 tries
    15:27:02.099 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 45
    15:27:02.099 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@45] cid=72938eb5-0625-4452-ae1b-d81b53103f14
    15:27:04.388 | s3209 WRITER   | DRAFT    86 words
    15:27:04.393 | s3209 WRITER   | SENT     to s3209_drafts cid=72938eb5-0625-4452-ae1b-d81b53103f14
    15:27:04.397 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 46
    15:27:04.399 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@46] cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa
    15:27:04.399 | s3209 WRITER   | RETRY    1/3 cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa TypeError: 'NoneType' objec
t is not iterable
    15:27:04.904 | s3209 WRITER   | RETRY    2/3 cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa TypeError: 'NoneType' objec
t is not iterable
    15:27:05.409 | s3209 WRITER   | RETRY    3/3 cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa TypeError: 'NoneType' objec
t is not iterable
    15:27:05.917 | s3209 WRITER   | DEAD_LETTER sent to s3209_dead_letter cid=e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa af
ter 3 tries
    15:27:05.919 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 47
    15:27:05.920 | s3209 WRITER   | RECEIVED from s3209_tasks [p0@47] cid=ae7bff49-09c2-4920-92c1-a8b6320e0961
    15:27:08.014 | s3209 WRITER   | DRAFT    86 words
    15:27:08.017 | s3209 WRITER   | SENT     to s3209_drafts cid=ae7bff49-09c2-4920-92c1-a8b6320e0961
    15:27:08.018 | s3209 WRITER   | COMMIT   s3209_tasks p0 offset -> 48
15:27:14.489 | s3209 TEST     | RESULT   good message drafted=1  poison in s3209_dead_letter=1
{
  "correlation_id": "e1d7deda-ebc4-4ced-aeb2-59b682b1f1fa",
  "error": "TypeError: 'NoneType' object is not iterable",
  "attempts": 3,
  "source": "s3209_tasks p0@46"
}
15:27:14.490 | s3209 TEST     | saved reports/hw08/raw/part3_poison_after.json
Ani Taraiya data260-3209 %
```

Poison after

The after run also cleared the poison left stuck by the before run (p0@44) and drafted the good message behind it (p0@45).

IV. Ordering

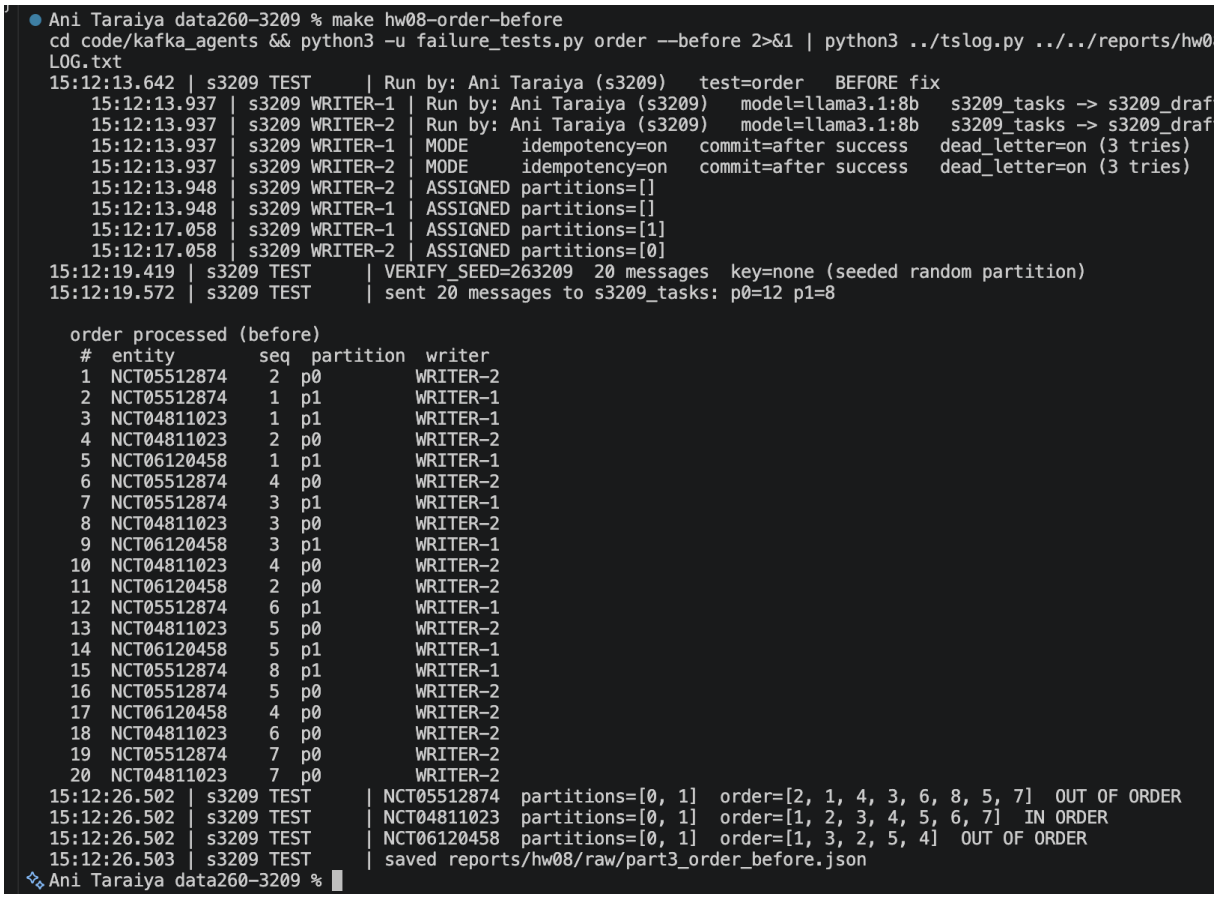
- **Test:** VERIFY_SEED 263209, 20 updates for 3 trials, s3209_tasks with 2 partitions, 2 Writers in group s3209_writer.
- **Fix:** message key = trial id, so one trial always maps to one partition.

Code Snippet:

```
251     for _ in range(20):
252         entity = rng.choice(entities)
253         seq[entity] += 1
254         msg = task(entity=entity, seq=seq[entity], work_ms=rng.randint(100, 600), run=label)
255         if before:
256             meta = send(msg, partition=rng.choice([0, 1]), quiet=True)
257         else:
258             meta = send(msg, key=entity, quiet=True)
259         sent.append((msg["correlation_id"], meta.partition))
260     log("TEST", f"sent 20 messages to {TASKS}: p0={sum(p == 0 for _, p in sent)} "
261                 f"p1={sum(p == 1 for _, p in sent)}")
```

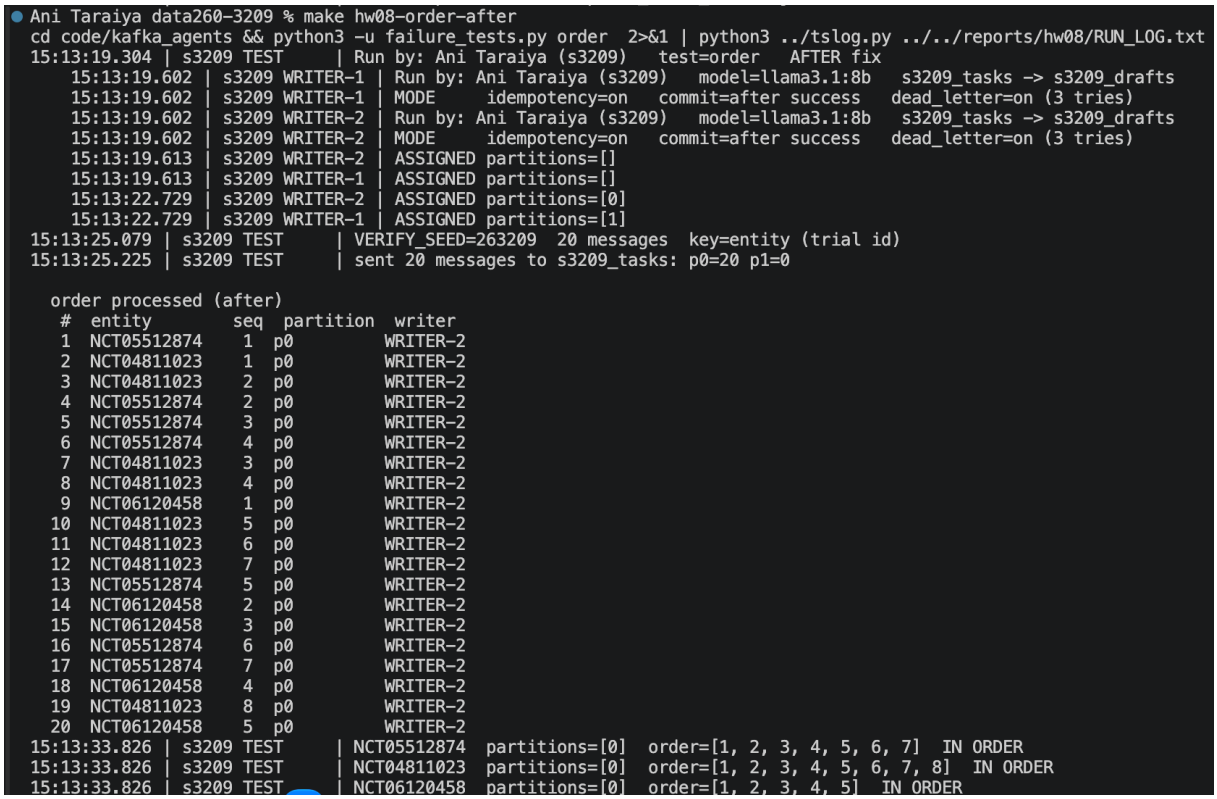
Partition key

Output Snippet:



Order before

Output Snippet:



```
Ani Taraiya data260-3209 % make hw08-order-after
cd code/kafka_agents && python3 -u failure_tests.py order  2>&1 | python3 ../tslog.py ../../reports/hw08/RUN_LOG.txt
15:13:19.304 | s3209 TEST     | Run by: Ani Taraiya (s3209)   test=order   AFTER fix
    15:13:19.602 | s3209 WRITER-1 | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
    15:13:19.602 | s3209 WRITER-1 | MODE     idempotency=on   commit=after success   dead_letter=on (3 tries)
    15:13:19.602 | s3209 WRITER-2 | Run by: Ani Taraiya (s3209)   model=llama3.1:8b   s3209_tasks -> s3209_drafts
    15:13:19.602 | s3209 WRITER-2 | MODE     idempotency=on   commit=after success   dead_letter=on (3 tries)
    15:13:19.613 | s3209 WRITER-2 | ASSIGNED partitions=[]
    15:13:19.613 | s3209 WRITER-1 | ASSIGNED partitions=[]
    15:13:22.729 | s3209 WRITER-2 | ASSIGNED partitions=[0]
    15:13:22.729 | s3209 WRITER-1 | ASSIGNED partitions=[1]
15:13:25.079 | s3209 TEST     | VERIFY_SEED=263209  20 messages  key=entity (trial id)
15:13:25.225 | s3209 TEST     | sent 20 messages to s3209_tasks: p0=20 p1=0

  order processed (after)
   #  entity       seq  partition  writer
   1  NCT05512874    1  p0         WRITER-2
   2  NCT04811023    1  p0         WRITER-2
   3  NCT04811023    2  p0         WRITER-2
   4  NCT05512874    2  p0         WRITER-2
   5  NCT05512874    3  p0         WRITER-2
   6  NCT05512874    4  p0         WRITER-2
   7  NCT04811023    3  p0         WRITER-2
   8  NCT04811023    4  p0         WRITER-2
   9  NCT06120458    1  p0         WRITER-2
  10  NCT04811023    5  p0         WRITER-2
  11  NCT04811023    6  p0         WRITER-2
  12  NCT04811023    7  p0         WRITER-2
  13  NCT05512874    5  p0         WRITER-2
  14  NCT06120458    2  p0         WRITER-2
  15  NCT06120458    3  p0         WRITER-2
  16  NCT05512874    6  p0         WRITER-2
  17  NCT05512874    7  p0         WRITER-2
  18  NCT06120458    4  p0         WRITER-2
  19  NCT04811023    8  p0         WRITER-2
  20  NCT06120458    5  p0         WRITER-2
15:13:33.826 | s3209 TEST     | NCT05512874  partitions=[0]  order=[1, 2, 3, 4, 5, 6, 7]  IN ORDER
15:13:33.826 | s3209 TEST     | NCT04811023  partitions=[0]  order=[1, 2, 3, 4, 5, 6, 7, 8]  IN ORDER
15:13:33.826 | s3209 TEST     | NCT06120458  partitions=[0]  order=[1, 2, 3, 4, 5]  IN ORDER
```

Order after

All three trial ids hashed to p0, so WRITER-2 did all the work. Order is kept per key; parallelism depends on how keys spread.

V. Failure Table

Failure scenario	Before fix	After fix
Duplicate delivery	2 deliveries -> 2 drafts, 2 final messages	2nd delivery: DUPLICATE skipped -> 1 draft, 1 final
Consumer killed mid-work	commit before work: p1 = 13 > msg 12 -> LOST, 0 drafts	commit after success: p1 = 13 = msg 13 -> redelivered, 1 draft, p1 = 14
Poison message	Writer crashed twice on p0@44, offset stuck, good msg 0 drafts	3 retries -> s3209_dead_letter, commit, good msg drafted
Two partitions / two consumers	no key: p0 = 12, p1 = 8; 2 of 3 trials out of order	key = trial id: one partition per trial, 3 of 3 in order

VI. Comparison with HW2

HW2 ran Planner and Reviewer as LangGraph function calls in one process.

Benefits gained

- **Decoupled failure:** killing the Writer did not stop Planner or Reviewer; work waited in s3209_tasks.
- **Independent scaling:** a second Writer joined the same group and took one partition.

New responsibilities

- **Duplicate delivery:** Kafka is at-least-once, so the Writer must be idempotent.
- **Ordering:** order holds only inside a partition, so the key must be chosen.
- **Also:** commit timing and poison handling are now our code, not the call stack.

Verification

- **Command:** make verify-hw08 -> reports/hw08/verification.json
- **Run on:** e5f85ee (code), llama3.1:8b, temperature 0, seed 3209, VERIFY_SEED 263209.
- **Scope:** starts its own agents, sends test messages only, never edits code.

Part	Checks
Part 1	containers, topics, CREATE, GET round trip, clean errors, FastAPI
Part 2	4 topics (tasks = 2 partitions), agents start, inbox -> final
Part 3	dead-letter topic, duplicate, commit after success, poison, key -> partition
Result	14 / 14 passed

Output Snippet:

```
Ani Taraiya data260-3209 % clear
make verify-hw08
15:44:38.999 | s3209 verify | SENT      to s3209_trial_requests cid=c1de4126-4de3-4e4f-864d-ec9aa43b0209 {"correlatio
n_id": "c1de4126-4de3-4e4f-864d-ec9aa43b0209", "reply_to": "s3209_trial_responses", "operation": "CREATE_TRIAL", "dat
a": {"trialTitle": "Verify Study of Drug Z in Hypertension", "sponsor": "SJSU Verify Lab", "email": "verify@sjsu-lab.
org", "content": "Smoke-test trial used by verify_hw08 to check the Kafka round trip.", "status": "Recruiting", "term
s": true}}
15:44:39.003 | s3209 verify | RESPONSE  from s3209_trial_responses cid=c1de4126-4de3-4e4f-864d-ec9aa43b0209 {"correla
tion_id": "c1de4126-4de3-4e4f-864d-ec9aa43b0209", "operation": "CREATE_TRIAL", "success": true, "trial_id": "aaa80500
-d021-46a7-a2d7-a62bd63c49ca", "message": "Trial created"}
[PASS] Part 1 CREATE_TRIAL over Kafka - success=True uuid trial_id=aaa80500-d021-46a7-a2d7-a62bd63c49ca cid kept=True
 (0.1s)
15:44:39.004 | s3209 verify | SENT      to s3209_trial_requests cid=007e7691-7edd-4ebf-909d-efe51b884ec0 {"correlatio
n_id": "007e7691-7edd-4ebf-909d-efe51b884ec0", "reply_to": "s3209_trial_responses", "operation": "GET_TRIAL", "data":
 {"trial_id": "aaa80500-d021-46a7-a2d7-a62bd63c49ca"}}
15:44:39.007 | s3209 verify | RESPONSE  from s3209_trial_responses cid=007e7691-7edd-4ebf-909d-efe51b884ec0 {"correla
tion_id": "007e7691-7edd-4ebf-909d-efe51b884ec0", "operation": "GET_TRIAL", "success": true, "trial": {"trial_id": "a
aa80500-d021-46a7-a2d7-a62bd63c49ca", "trialTitle": "Verify Study of Drug Z in Hypertension", "sponsor": "SJSU Verify
 Lab", "email": "verify@sjsu-lab.org", "content": "Smoke-test trial used by verify_hw08 to check the Kafka round trip
.", "status": "Recruiting", "terms": true}}
[PASS] Part 1 GET_TRIAL round trip - success=True fields match=True cid kept=True (0.0s)
15:44:39.010 | s3209 verify | SENT      to s3209_trial_requests cid=1b95a83c-2052-4a17-8f5a-3a4d1b31d0b3 {"correlatio
n_id": "1b95a83c-2052-4a17-8f5a-3a4d1b31d0b3", "reply_to": "s3209_trial_responses", "operation": "GET_TRIAL", "data":
 {"trial_id": "41fcd208-3d59-420e-8da8-e428ba42a21c"}}
15:44:39.013 | s3209 verify | RESPONSE  from s3209_trial_responses cid=1b95a83c-2052-4a17-8f5a-3a4d1b31d0b3 {"correla
tion_id": "1b95a83c-2052-4a17-8f5a-3a4d1b31d0b3", "operation": "GET_TRIAL", "success": false, "error": "Trial not fou
nd"}
15:44:39.015 | s3209 verify | SENT      to s3209_trial_requests cid=44ece3d9-7683-499e-9f43-88c932112c06 {"correlatio
n_id": "44ece3d9-7683-499e-9f43-88c932112c06", "reply_to": "s3209_trial_responses", "operation": "DELETE_TRIAL", "dat
a": {}}
15:44:39.017 | s3209 verify | RESPONSE  from s3209_trial_responses cid=44ece3d9-7683-499e-9f43-88c932112c06 {"correla
tion_id": "44ece3d9-7683-499e-9f43-88c932112c06", "operation": "DELETE_TRIAL", "success": false, "error": "Unknown op
eration"}
15:44:39.018 | s3209 verify | SENT      to s3209_trial_requests cid=05c571e6-71f1-4153-bd58-f3e4c19fd68d {"correlatio
n_id": "05c571e6-71f1-4153-bd58-f3e4c19fd68d", "reply_to": "s3209_trial_responses", "operation": "CREATE_TRIAL", "dat
a": {"trialTitle": "Verify Study of Drug Z in Hypertension", "sponsor": "SJSU Verify Lab", "email": "bad", "content":
 "short", "status": "Recruiting", "terms": true}}
15:44:39.020 | s3209 verify | RESPONSE  from s3209_trial_responses cid=05c571e6-71f1-4153-bd58-f3e4c19fd68d {"correla
tion_id": "05c571e6-71f1-4153-bd58-f3e4c19fd68d", "operation": "CREATE_TRIAL", "success": false, "error": "Validation
 failed: 'email' must be a valid email address; 'content' must be more than 25 characters"}
[PASS] Part 1 clean errors keep correlation_id - not found / unknown op / invalid data -> clean error + cid kept: [Tr
ue, True, True] (0.0s)
[PASS] Part 1 FastAPI wrapper on PORT_BASE - POST/GET via http://127.0.0.1:8509: success=True header cid kept=True (0
.0s)
[PASS] Part 2 four topics, tasks has 2 partitions - s3209_inbox=1, s3209_tasks=2, s3209_drafts=1, s3209_final=1 (0.1s
)
[PASS] Part 2 planner, writer, reviewer start - planner/writer/reviewer ready=[True, True, True] writer owns both tas
k partitions=True (2.4s)
[PASS] Part 2 question reaches s3209_final - inbox -> final: status=approved answer words=79 cid kept=True (7.4s)
[PASS] Part 3 dead-letter topic exists - s3209_dead_letter partitions=1 (0.1s)
[PASS] Part 3 duplicate replay has no extra draft - same message delivered twice -> drafts=1 (want 1) (11.6s)
[PASS] Part 3 offset committed after success - writer committed p0=51 after last message at offset 50 (0.0s)
[PASS] Part 3 poison -> dead letter, next message runs - poison in s3209_dead_letter=1 attempts=3; next message draft
ed=True (6.0s)
[PASS] Part 3 same key -> same partition - key=trial id -> partitions per trial: {'NCT05512874': [0], 'NCT04811023':
[0], 'NCT06120458': [0]} (0.1s)

14/14 checks passed -> reports/hw08/verification.json
```

Verification run

AI Use

1. What I used an AI assistant for, and what I did myself

I used an AI assistant to understand the concepts in this homework (request/response over Kafka with a correlation_id, consumer groups and partitions, when to commit offsets, idempotency, dead-letter topics and message keys) and to help me write the code for each part, the failure-test and verification scripts and the report layout. I did not trust it blindly: I asked questions until I understood each concept and what each script was doing.

I did the setup and running myself. I ran the Docker stack (Kafka, worker, FastAPI), Ollama, the three agents in separate terminals, and every demo, failure test and verification run on my Mac. I took the screenshots and checked each output against the assignment before it went into the report.

2. One AI-produced output that was wrong

The first version of the Part 3 failure tests assumed the test's Writer was the only member of the s3209_writer consumer group. The Part 2 Writer was still running in another terminal, so the two Writers split the s3209_tasks partitions. The duplicate "after fix" run crashed with TypeError: unsupported operand type(s) for +: 'NoneType' and 'int' because the message went to the other Writer, and the duplicate, crash and poison runs mixed output from both Writers.

3. How I detected the problem

In RUN_LOG.txt the test Writer logged ASSIGNED partitions=[1] instead of [0, 1], and there were WRITER lines without the test's indent, which meant a second Writer was consuming. The poison "before fix" run also printed a traceback from the old Part 2 code path.

4. What I changed and why it works now

The test now checks that its Writer owns both partitions before sending anything, and stops with a clear message if another Writer is in the group. The None case in the wait was fixed. I stopped the Part 2 Writer and re-ran the six tests (15:19-15:27): every run logged ASSIGNED partitions=[0, 1], the results match the expected before/after behaviour, and make verify-hw08 passed 14 / 14 checks.